

claude-windows / edc346c2-92fc-4ad1-b556-c7ab955a5613

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\edc346c2-92fc-4ad1-b556-c7ab955a5613\subagents\workflows\wf_47347652-5ae\agent-a788b1052b4173b03.jsonl`
- SHA-256: `6f7a365c299b1adc9a573719c4a7c48e560aa5f72d7b530897e2b9e775c8b2cc`
- Source modified: `2026-07-05T06:59:34+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_47347652-5ae`
- session_id: `edc346c2-92fc-4ad1-b556-c7ab955a5613`
```

Transcript

1. user (2026-07-05T06:56:46.210Z)

SHARED CONTEXT (verified live this session, 2026-07-05):

```
- Repos (local checkouts = current master):
  * ENGINE: C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer (GitHub Khelsutra/badminton-highlight-indexer)
  * SPA: C:\Users\avidu\Projects\khelsutra-guru\khelsutra (web/src is the Studio SPA; GitHub avidullu/khelsutra)
- The LIVE hosted control-plane is an OLDER build than local master: capabilities reports engine_version "0.1.0",
  and its capabilities JSON has NO max_upload_mb / max_part_mb fields (current master DOES expose them, added around khelsutra #153). So distinguish "current master code" (what you can read) from "deployed old-build behavior" and
  note where they likely diverge.
- Deployed control-plane config.json (base64-injected at boot) sets ONLY:
  {"deployment":{"profile":"cloud-serving","compute_target":"cloud_run"},"indexing":{"skip_ai_handoff":true},
  "storage":{"backend":"gcs","output_root":"gs://khelsutra-rally-serving","input_backend":"gcs",
  "input_root":"gs://khelsutra-rally-serving/videos"},"security":{"...edge auth...}}
  Note: it does NOT pin any default/segmenter, and does NOT override security.max_upload_mb or max_part_mb.
- Live capabilities from the deployed CP: default_segmenter="yolo_hybrid",
  wasb_weights_present=false,
  gpu.nvidia_smi_present=false, gemini_key_present=false, deployment.cloud_available=true,
  profile=cloud-serving.
- Architecture: the CP (Cloud Run service, 4Gi RAM, no GPU, /tmp is RAM-backed) serves the SPA +
  dispatches
  cloud jobs to an L4 GPU Cloud Run JOB "rally-worker" which HAS the WASB weights mounted
  (WASB_WEIGHTS=/gcs/models/...).
- Config defaults known from code: SecurityConfig.max_upload_mb=4096 (MB whole-file),
  max_part_mb=95 (per part).
- The e2e/battery that validated the alpha ALWAYS passed segmenter:"wasb_hybrid" EXPLICITLY in
  /api/process.
```

A finder proposed this answer to ISSUE 2 (does a tester cloud job run wasb_hybrid or yolo_hybrid?):

```
{
  "root_cause": "When a tester ingests a video through the SPA with compute=\"cloud\", the
  dispatched L4 rally-worker job today runs yolo_hybrid, NOT the validated wasb_hybrid. The SPA
  never sends a \"segmenter\" field, so req.segmenter arrives as None on the control-plane. The CP
  resolves seg_name = req.segmenter or default_seg, where default_seg =
  indexing.default_segmenter. The deployed config.json does NOT pin indexing.default_segmenter (it
  sets only skip_ai_handoff under indexing), so the code default \"yolo_hybrid\""
```

(IndexingConfig.default_segmenter) wins. The CP then forwards this resolved seg_name EXPLICITLY to the worker as "--segmenter yolo_hybrid", so the worker's own default (which would resolve to its config's default_segmenter if the arg were absent) is never reached ? the CP-resolved value is authoritative. This is a DEPLOYED-CONFIG gap (config.json omits the segmenter pin), not a code bug.",

```
"evidence": [
  {
    "file": "khelsutra/web/src/api/client.ts",
    "line": 100,
    "detail": "processVideo body is JSON.stringify({ video_path: videoPath, locale, compute })
? no 'segmenter' field is ever sent. JSON.stringify drops undefined, so the engine receives
req.segmenter = None."
  },
  {
    "file": "khelsutra/web/src/hooks/useIngestJob.ts",
    "line": 259,
    "detail": "The only real caller: processVideo(uri, cid, locale, compute) ? 4 positional
args, no segmenter. IngestPanel.tsx has a local/cloud compute-picker (line 214) but NO segmenter
picker; grep for 'segmenter' in web/src hits only types.ts (caps shape) and SetupWizard.tsx
(display), never the ingest/process path."
  },
  {
    "file": "badminton-highlight-indexer/backend/config/models.py",
    "line": 181,
    "detail": "IndexingConfig.default_segmenter: str = \"yolo_hybrid\" ? the code default.
Deployed config.json sets indexing.skip_ai_handoff=true but does NOT set
indexing.default_segmenter, so this default stands."
  },
  {
    "file": "badminton-highlight-indexer/backend/main.py",
    "line": 964,
    "detail": "The /api/process handler (process_video) persists req.segmenter verbatim into
the job (create_job(..., req.segmenter, ...)) ? which is None. It does NOT apply the default
here; the default is applied later in the drain/worker orchestration."
  },
  {
    "file": "badminton-highlight-indexer/backend/orchestration.py",
    "line": 476,
    "detail": "In the CP drain: default_seg = getattr(indexing, 'default_segmenter', 'motion');
seg_name = req.segmenter or default_seg. With req.segmenter=None and deployed
default_segmenter='yolo_hybrid', seg_name resolves to 'yolo_hybrid'. This seg_name is passed to
_maybe_dispatch_remote(...)."
  },
  {
    "file": "badminton-highlight-indexer/backend/orchestration.py",
    "line": 341,
    "detail": "The cloud dispatch builds ComputeJobSpec(video_uri=..., out_uri=...,
segmenter=seg_name, config_overrides=(skip_ai_handoff only)). seg_name ('yolo_hybrid') is
forwarded to the worker as the segmenter. NOTE: config_overrides forwards only
indexing.skip_ai_handoff (lines 334-337); it does NOT forward the segmenter via a config
override ? the segmenter travels via spec.segmenter ? --segmenter."
  },
  {
    "file": "badminton-highlight-indexer/backend/compute/cloud_run.py",
    "line": 192,
    "detail": "run_infer builds worker argv: 'if spec.segmenter: argv += [\"--segmenter\",
spec.segmenter]'. Since seg_name is always truthy, the worker is invoked with an EXPLICIT '--
segmenter yolo_hybrid'."
  },
  {
    "file": "badminton-highlight-indexer/backend/worker/__main__.py",
    "line": 256,
    "detail": "Worker: seg_name = args.segmenter or config.indexing.default_segmenter. Because
the CP always passes --segmenter explicitly (default arg is None, line 651),
args.segmenter='yolo_hybrid' and the worker's own config default fallback is never exercised.
```

The worker HAS wasb weights mounted but is nonetheless told to run yolo_hybrid."

```
}
1,
"deployed_vs_master": "The trace holds identically for both the deployed OLD build
(engine_version 0.1.0) and current master for this specific question. The default-segmenter
resolution order (SPA omits segmenter ? CP resolves req.segmenter or indexing.default_segmenter
? forwards explicit --segmenter to worker) predates the max_upload_mb/max_part_mb additions
(khelsutra #153) and is unchanged. The live capabilities already prove the deployed
default_segmenter='yolo_hybrid' (main.py capabilities handler reads
getattr(indexing,'default_segmenter',None), and the deployed config leaves it at the
'yolo_hybrid' code default). The only divergence relevant here is unrelated: current master
exposes max_upload_mb/max_part_mb, the deployed build does not ? that does not affect segmenter
selection. Caveat I could not verify live: I read the deployed config.json only from the shared
session context (not fetched fresh this trace), but the live capabilities value
default_segmenter='yolo_hybrid' + wasb_weights_present=false is exactly consistent with
'config.json does not pin the segmenter'.",
"confidence": "high",
"fix_options": [
  "PREFERRED (deploy-only, no code/redeploy of images): pin the segmenter in the CP's
base64-injected config.json by adding indexing.default_segmenter='wasb_hybrid'. Then
req.segmenter(None) or default_seg ? 'wasb_hybrid', the CP forwards --segmenter wasb_hybrid to
the L4 worker (which has WASB weights), and capabilities will also start reporting
default_segmenter='wasb_hybrid' (self-consistent). Single-line config change; lowest blast
radius.",
  "SPA sends segmenter explicitly: have processVideo/IngestPanel pass segmenter='wasb_hybrid'
(or add a segmenter picker defaulting to wasb_hybrid). This makes req.segmenter truthy so it
wins regardless of server default. Requires an SPA build+deploy and leaves the server default
still 'yolo_hybrid' (capabilities would still report yolo_hybrid ? cosmetically inconsistent),
so it's weaker alone than the config pin.",
  "Belt-and-suspenders: do BOTH ? pin indexing.default_segmenter='wasb_hybrid' in CP
config.json AND have the SPA send segmenter explicitly ? so neither a config regression nor an
SPA regression can silently drop back to the unvalidated yolo_hybrid.",
  "OPTIONAL guard (code, defense-in-depth for the future 'validated segmenter' contract): in
the cloud dispatch path, refuse/annotate dispatch when the resolved seg_name is not in the
validated set for cloud serving, so an unvalidated segmenter can't be silently dispatched to the
paid L4 worker. Not required to fix today's gap; the config pin resolves it."
],
"open_questions": [
  "Is running yolo_hybrid on the L4 worker actually harmful, or merely 'unvalidated'? The
worker has WASB weights mounted (WASB_WEIGHTS=/gcs/models/...) but yolo_hybrid does not use
them; need to confirm yolo_hybrid even has functioning weights/behavior on the worker image,
else cloud jobs may produce degraded/zero-rally results rather than just 'a different validated-
vs-unvalidated path'.",
  "Should the capabilities-reported default_segmenter and the segmenter actually dispatched be
treated as a single source of truth? Today capabilities='yolo_hybrid' happens to match what's
dispatched ? pinning via config keeps them consistent; fixing only in the SPA would make
capabilities lie about what cloud jobs run.",
  "Confirm no OTHER deployed override (env var or a config layer not in the shared config.json
snapshot) sets indexing.default_segmenter on the live CP ? the live capabilities value
'yolo_hybrid' strongly implies none does, but a fresh fetch of the running config would make
this airtight."
]
}
```

Independently re-check. The launch review + copy assume WASB-only, so a false "it's fine" OR a false alarm both matter. Verify against actual code: (a) does the SPA truly omit/include segmenter in /api/process (khelsutra/web/src/api/client.ts, IngestPanel.tsx)? (b) does the cloud dispatch forward segmenter and what does the worker default to (backend/orchestration.py, backend/worker/__main__.py, config default resolution)? Try to REFUTE the finder's NET verdict. Return CONFIRMED / PARTLY_CONFIRMED / REFUTED with your own file:line evidence.

2. assistant (2026-07-05T06:56:52.078Z)

I'll independently verify the finder's claim by tracing the actual code. Let me start by pulling the repos fresh and examining the key files.

3. user (2026-07-05T06:56:58.504Z)

Already up to date.

---SPA---

Already up to date.

4. user (2026-07-05T06:56:59.217Z)

64fa4a2 docs(a13): alpha launch review ? conditional-GO recommendation; GO/NO-GO to owner (#478)

---SPA---

5ddb81b Fix hosted-alpha copy blockers: honest cloud disclosure in wizard, ingest, privacy policy (#154) (#155)

5. assistant (2026-07-05T06:57:02.463Z)

Let me examine the actual code files cited in the finder's evidence, starting with the SPA and then the backend.

6. user (2026-07-05T06:57:03.570Z)

```
1      // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2      // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3      // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type {
6          RallySegment,
7          QueryResponse,
8          CompileResult,
9          EngineConfig,
10         EngineCapabilities,
11         ProcessResponse,
12         JobStatus,
13         UploadInitResponse,
14         UploadPartResponse,
15         UploadCompleteResponse,
16         VideoSummary,
17         VideosResponse,
18         AnnotationsResponse,
19         AssetResponse,
20     } from "../types";
21
22     /** Resolve the engine API base. Default same-origin `/api`. For a SPLIT deploy ? SPA
on
23     * Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md §2) ?
24     * set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
25     * (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
26     * Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
27     * `` `${base}/process` `` never double-slashes. */
28     export function apiBase(): string {
29         return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
30     }
31
32     const BASE = apiBase();
33
```

```

34     export class ApiError extends Error {
35         readonly cid?: string;
36         readonly status?: number;
37         readonly body?: string;
38         constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
39             super(message);
40             this.name = "ApiError";
41             this.cid = opts.cid;
42             this.status = opts.status;
43             this.body = opts.body;
44             if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
45         }
46     }
47
48     interface RequestOpts {
49         method?: string;
50         body?: string;
51         headers?: Record<string, string>;
52         /** Correlation id to thread through this call; auto-generated if absent. */
53         cid?: string;
54     }
55
56     async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
57         const cid = opts.cid ?? newCorrelationId();
58         const url = `${BASE}${path}`;
59         const method = opts.method ?? "GET";
60         log("info", "api.request", { cid, method, url });
61
62         let res: Response;
63         try {
64             res = await fetch(url, {
65                 method,
66                 body: opts.body,
67                 headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
68             });
69         } catch (err) {
70             // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
71             log("error", "api.network_error", { cid, method, url, error: String(err) });
72             throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
73         }
74
75         if (!res.ok) {
76             const body = await res.text().catch(() => "");
77             log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
78             throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
79         }
80
81         const data = (await res.json()) as T;
82         log("info", "api.response", { cid, method, url, status: res.status });
83         return data;
84     }
85
86     /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
87     * processing. POST /api/process {video_path} ? 202 {job_id, ...} [engine process
route]. Fast-fails 400
88     * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
89     export async function processVideo(
90         videoPath: string,
91         cid?: string,

```

```

92     locale?: string,
93     compute?: string,
94   ): Promise<ProcessResponse> {
95     // `locale` (the UI language) is recorded for PMF analytics (which markets use the
96     // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the
server
97     // default. Both optional + omitted when absent (JSON.stringify drops undefined).
98     return request<ProcessResponse>("/process", {
99       method: "POST",
100       body: JSON.stringify({ video_path: videoPath, locale, compute }),
101       cid,
102     });
103   }
104
105   /** Poll a submitted job. GET /api/jobs/{job_id} [engine jobs route]. 404 ? unknown
job_id. */
106   export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
107     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
108   }
109
110   /** List indexed videos, newest first ? the "my videos" library. GET /api/videos [engine
list_videos route].
111   * Reload-survival: a past match can be reopened without re-processing. Returns [] on
an empty box. */
112   export async function listVideos(cid?: string): Promise<VideoSummary[]> {
113     const data = await request<VideosResponse>("/videos", { method: "GET", cid });
114     return data.videos ?? [];
115   }
116
117   /** The source-video stream URL for annotation playback. GET /api/source/{video_id}
(P8). The engine
118   * serves a frame-identical proxy; labels stay keyed to the master's MD5
(STUDIO_MEDIA_LIFECYCLE D4). */
119   export function sourceUrl(videoId: string): string {
120     return `${BASE}/source/${encodeURIComponent(videoId)}`;
121   }
122
123   /** Persist finished rally labels as the annotator CSV. POST /api/annotations {video_id,
csv} (P8).
124   * Sent as the byte-compatible CSV (serializeRows) so the engine writes it verbatim
into the corpus. */
125   export async function postAnnotations(videoId: string, csv: string, cid?: string):
Promise<AnnotationsResponse> {
126     return request<AnnotationsResponse>("/annotations", {
127       method: "POST",
128       body: JSON.stringify({ video_id: videoId, csv }),
129       cid,
130     });
131   }
132
133   /** Store a video into the user's chosen medium + register it as a library asset (P4,
D14).
134   * POST /api/assets {uploaded_path? | source_uri?, medium_uri, sport?} ? 201 {video_id,
stored_uri}.
135   * Exactly ONE source: an `uploadedPath` (a just-uploaded local file ? a durable
MD5-keyed copy is
136   * put into the medium, the local upload kept as the working copy) OR a `sourceUri` (a
video already
137   * at rest, registered as-is). Feed `stored_uri` into processVideo(). Fast-fails 400
(bad medium /
138   * both-or-neither source), 403 (hosted posture), 404 (missing file), 507 (no space),
502 (put) ?
139   * surfaced as ApiError. */

```

```

140     export async function createAsset(
141         args: { uploadedPath?: string; sourceUri?: string; mediumUri: string; sport?: string
142     },
143     cid?: string,
144 ): Promise<AssetResponse> {
145     return request<AssetResponse>("/assets", {
146         method: "POST",
147         body: JSON.stringify({
148             uploaded_path: args.uploadedPath,
149             source_uri: args.sourceUri,
150             medium_uri: args.mediumUri,
151             sport: args.sport,
152         }),
153         cid,
154     });
155 }
156 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
157 [engine query route]. */
158 export async function listRallies(videoId: string, cid?: string):
159 Promise<RallySegment[]> {
160     const data = await request<QueryResponse>("/query", {
161         method: "POST",
162         body: JSON.stringify({ video_id: videoId }),
163         cid,
164     });
165     return data.play_segments ?? [];
166 }
167 // Compile-job poll cadence (mirrors useIngestJob): a long ffmpeg stitch must not block
168 the request.
169 const COMPILE_POLL_MIN_MS = 1000;
170 const COMPILE_POLL_MAX_MS = 5000;
171 const COMPILE_POLL_BACKOFF = 1.5;
172 const COMPILE_POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' compile won't poll
173 forever
174 /** Stitch selected rallies into one reel. ASYNC (#329): POST /api/compile returns 202 +
175 a job_id;
176 * the ffmpeg stitch runs on the worker and we POLL GET /api/jobs/{id} for the reel ? a
177 long stitch
178 * would otherwise exceed the ~100 s edge/proxy timeout (524) while the reel actually
179 built. Resolves
180 * with the job's CompileResult on success; throws ApiError on a failed/timed-out
181 compile.
182 * `locale` (the UI language) localizes the reel watermark engine-side (omitted when
183 absent). */
184 export async function compileReel(
185     videoId: string,
186     segmentIds: number[],
187     outputFilename: string,
188     cid?: string,
189     locale?: string,
190 ): Promise<CompileResult> {
191     const c = cid ?? newCorrelationId();
192     const submitted = await request<ProcessResponse>("/compile", {
193         method: "POST",
194         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
195 outputFilename, locale }),
196         cid: c,
197     });
198     return pollCompileJob(submitted.job_id, c);
199 }
200 /** Poll a submitted compile job to its terminal state. Returns the reel result on

```

```

success; throws an
194     * ApiError for an engine job-failure, a success-shaped FAILURE (done but the stitch
failed / no URL),
195     * or a poll timeout ? so the caller's existing classifyError/errorText mapping
surfaces it like ingest. */
196     async function pollCompileJob(jobId: string, cid: string): Promise<CompileResult> {
197         const startedAt = Date.now();
198         let delay = COMPILE_POLL_MIN_MS;
199         for (;;) {
200             const job = await getJob(jobId, cid);
201             if (job.status === "done") {
202                 const result = (job.result ?? {}) as CompileResult;
203                 if (result.status === "failed" || !result.download_url) {
204                     // A real stitch failure carries an engine message ? surface it as an `engine`
detail (status +
205                     // body set) so classifyError shows it verbatim, NOT the no-status "engine
unreachable" fallback.
206                     const msg = result.message || "Highlight stitching failed.";
207                     throw new ApiError(msg, { cid, status: 502, body: msg });
208                 }
209                 return result;
210             }
211             if (job.status === "failed") {
212                 const msg = job.error || "Compile job failed.";
213                 throw new ApiError(msg, { cid, status: 502, body: msg });
214             }
215             if (Date.now() - startedAt > COMPILE_POLL_CEILING_MS) {
216                 throw new ApiError("Compile timed out.", { cid, status: 504, body: "Compile timed
out." });
217             }
218             await new Promise((resolve) => setTimeout(resolve, delay));
219             delay = Math.min(delay * COMPILE_POLL_BACKOFF, COMPILE_POLL_MAX_MS);
220         }
221     }
222
223     /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [engine highlights route]. */
224     export function highlightUrl(videoId: string, filename: string): string {
225         return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
226     }
227
228     /** Read engine config (for the stitching floor). GET /api/config [engine config route].
*/
229     export async function getConfig(cid?: string): Promise<EngineConfig> {
230         return request<EngineConfig>("/config", { method: "GET", cid });
231     }
232
233     /** What THIS box can do ? drives the first-run wizard. GET /api/capabilities
(PACKAGING_PLAN P2c). */
234     export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
235         return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
236     }
237
238     /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[engine compile floor]. */
239     export function minShotCount(cfg: EngineConfig): number | undefined {
240         return cfg.stitching?.min_shot_count;
241     }
242
243     // ?? chunked upload (backend/api/uploads.py) ?????????????????????????????????????
244     // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
245     // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.

```



```

246
247     /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[backend/api/uploads.py]. */
248     export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
249         return request<UploadInitResponse>("/upload/init", {
250             method: "POST",
251             body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
252             cid,
253         });
254     }
255
256     /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[backend/api/uploads.py]. */
257     export async function uploadPart(
258         uploadId: string,
259         index: number,
260         chunk: Blob,
261         cid?: string,
262         signal?: AbortSignal,
263     ): Promise<UploadPartResponse> {
264         const c = cid ?? newCorrelationId();
265         const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
266         log("info", "api.request", { cid: c, method: "PUT", url });
267         let res: Response;
268         try {
269             res = await fetch(url, {
270                 method: "PUT",
271                 body: chunk,
272                 headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
273                 signal,
274             });
275         } catch (err) {
276             log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
});
277             throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
278         }
279         if (!res.ok) {
280             const body = await res.text().catch(() => "");
281             log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
282             throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
283         }
284         log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
285         return (await res.json()) as UploadPartResponse;
286     }
287
288     /** Finalize: returns {path, video_id} ? feed `path` to processVideo()
[backend/api/uploads.py]. */
289     export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
290         return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
291             method: "POST",
292             body: JSON.stringify({ total_parts: totalParts }),
293             cid,
294         });
295     }
296
297     /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[backend/api/uploads.py]. */
298     export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
299         await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });

```

```

300     }
301
302     export interface UploadProgress {
303         sentBytes: number;
304         totalBytes: number;
305     }
306
307     /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
308     * On ANY failure or abort it best-effort DELETES the server-side partial, then
rethrows so the
309     * caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}.
310     * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
recovery is a
311     * fresh init (re-pick the file), which the UI's retry does. */
312     export async function uploadFile(
313         file: File,
314         opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {},
315     ): Promise<UploadCompleteResponse> {
316         const cid = opts.cid ?? newCorrelationId();
317         const init = await uploadInit(file.name, file.size, cid);
318         // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
319         // would silently upload zero parts) ? fall back to the engine's 95 MB default.
320         const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
321         const partBytes = partMb * 1024 * 1024;
322         const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
323         log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
324         let sent = 0;
325         try {
326             for (let i = 0; i < totalParts; i++) {
327                 if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
328                 const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
329                 await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
330                 sent += chunk.size;
331                 opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
332             }
333             const done = await uploadComplete(init.upload_id, totalParts, cid);
334             log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
335             return done;
336         } catch (err) {
337             log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
338             await uploadAbort(init.upload_id, cid).catch(() => {});
339             throw err;
340         }
341     }
342

```

7. user (2026-07-05T06:57:04.278Z)

```

240         const run: Run = { cancelled: false };
241         runRef.current = run;
242
243         const cid = newCorrelationId();
244         const startedAt = Date.now();
245         setPhase({ status: "submitting", ...(displayName ? { displayName } : {}) });
246         log("info", "ingest.submit_start", { cid, uri, display_name: displayName });
247
248         const failSubmit = (error: IngestError, event: string) => {

```

```

249         clearActiveJob();
250         replaceJobInUrl(null);
251         setPhase({ status: "error", error });
252         const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
: "error";
253         log(level, event, { cid, uri, code: error.code, detail: error.detail });
254     };
255
256     void (async () => {
257         let jobId: string;
258         try {
259             const resp = await processVideo(uri, cid, locale, compute);
260             jobId = resp.job_id;
261         } catch (err) {
262             failSubmit(classifyError(err), "ingest.submit_error");
263             return;
264         }
265         if (run.cancelled) return;
266         const active: ActiveIngestJob = { jobId, uri, cid, startedAt, locale, compute,
displayName };
267         writeActiveJob(active);
268         replaceJobInUrl(jobId);
269         log("info", "ingest.queued", { cid, uri, job_id: jobId });
270         startPolling(active, { initialDelay: POLL_MIN_MS });
271     })();
272 },
273 [startPolling, stop],
274 );
275
276 // On a fresh page load, resume either the explicit URL job or the most recent saved
one.
277 useEffect(() => {
278     if (autoResumeAttemptedRef.current) return;
279     autoResumeAttemptedRef.current = true;

```

8. assistant (2026-07-05T06:57:11.542Z)

The SPA side is confirmed: `processVideo(uri, cid, locale, compute)` is called with 4 args, no segmenter, and the body is `JSON.stringify({ video\_path, locale, compute })` with no segmenter field. Let me verify there's no other caller of processVideo and no segmenter picker, then trace the backend.

9. user (2026-07-05T06:57:13.665Z)

```

C:\Users\avidu\Projects\kheIsutra-guru\kheIsutra\web\src\api\types.ts:20:  /** motion-segmenter
FABRICATION (random) ? DO NOT surface (see PER_RALLY_SELECTION.md). */
C:\Users\avidu\Projects\kheIsutra-guru\kheIsutra\web\src\api\types.ts:22:  /** motion-segmenter
FABRICATION (random) ? DO NOT surface. */
C:\Users\avidu\Projects\kheIsutra-guru\kheIsutra\web\src\api\types.ts:95:  segmenters?:
string[];
C:\Users\avidu\Projects\kheIsutra-guru\kheIsutra\web\src\api\types.ts:96:  default_segmenter?:
string;
C:\Users\avidu\Projects\kheIsutra-guru\kheIsutra\web\src\components\SetupWizard.tsx:54:  const
detector = caps?.default_segmenter ?? "motion";
C:\Users\avidu\Projects\kheIsutra-guru\kheIsutra\web\src\components\SetupWizard.test.tsx:24:
default_segmenter: "motion",
C:\Users\avidu\Projects\kheIsutra-guru\kheIsutra\web\src\components\SetupWizard.test.tsx:30:
default_segmenter: "gemini",
C:\Users\avidu\Projects\kheIsutra-guru\kheIsutra\web\src\components\SetupWizard.test.tsx:38:
default_segmenter: "wasb_hybrid",

```

10. user (2026-07-05T06:57:14.424Z)

```

C:\Users\avidu\Projects\kheIsutra-guru\kheIsutra\web\src\api\client.ts:89:export async function

```

```

processVideo(
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:137: * at rest,
registered as-is). Feed `stored_uri` into processVideo(). Fast-fails 400 (bad medium /
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:288:/** Finalize: returns
{path, video_id} ? feed `path` to processVideo() [backend/api/uploads.py]. */
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useIngestJob.ts:10:import {
getJob, processVideo } from "../api/client";
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useIngestJob.ts:259:
const resp = await processVideo(uri, cid, locale, compute);
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\types.ts:140: /** The durable,
MD5-keyed location ? pass as video_path to processVideo(). */
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useIngestJob.test.ts:38: await
vi.advanceTimersByTimeAsync(0); // flush processVideo ? schedule the first poll
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\locale.test.ts:5:import {
compileReel, processVideo } from "../client";
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\locale.test.ts:52:
it("processVideo includes the locale when given, omits it when absent", async () => {
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\locale.test.ts:55: await
processVideo("gs://b/x.mp4", "cid", "hi");
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\locale.test.ts:58: await
processVideo("gs://b/y.mp4", "cid");

```

11. assistant (2026-07-05T06:57:21.188Z)

SPA confirmed: the only real caller is `useIngestJob.ts:259` with no segmenter; `segmenter` appears in the SPA only in the capabilities-shape type (`types.ts`) and display (`SetupWizard.tsx`), never in the ingest/process path. Now let me trace the backend. Let me look at the process handler, the request model, and the orchestration path.

12. user (2026-07-05T06:57:22.201Z)

```

940         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
941             raise HTTPException(
942                 status_code=404, detail=f"Video file not found at '{req.video_path}'"
943             )
944         # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
945         # if this box can't hold that copy. Best-effort ? an unstatatable remote
(network/SDK) or an
946         # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
947         # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
948         if input_ref.backend != "local":
949             need = _remote_input_size_bytes(
950                 input_ref, getattr(request.app.state.config, "storage", None)
951             )
952             require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
953         if req.segmenter and not segmenter_registry.get(req.segmenter):
954             available = list(segmenter_registry.list_available().keys())
955             raise HTTPException(
956                 status_code=400,
957                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
958             )
959
960         job_id = uuid.uuid4().hex
961         if not request.app.state.db.create_job(
962             job_id,
963             req.video_path,
964             req.segmenter,
965             req.player_pool,
966             req.max_frames,
967             owner_id=owner_id,
968             locale=req.locale,

```

```

969         compute=req.compute,
970         force=req.force,
971     ):
972         raise HTTPException(status_code=500, detail="Failed to persist job record.")
973     _get_executor().submit(
974         _drain_due_jobs, request.app.state.config, request.app.state.db
975     )
976     return JSONResponse(
977         status_code=202,
978         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
979     )
980
981
982     @app.get("/api/jobs/{job_id}/cost")
983     def get_job_cost(job_id: str, request: Request):
984         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
985         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
986         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
987         cost = request.app.state.db.get_job_cost(job_id)
988         if cost is None:
989             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
990         fx = request.app.state.config.billing.fx_inr_per_usd
991         usd = cost.get("cost_usd")
992         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
993         cost["fx_inr_per_usd"] = fx
994         return cost
995
996
997     @app.get("/api/videos/{video_id}/cost")
998     def get_video_cost(video_id: str, request: Request):
999         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the

```

13. user (2026-07-05T06:57:23.293Z)

```

320         "? that's where the Cloud Run job writes the result.",
321         False,
322     )
323
324     notify("dispatching (cloud_run)")
325     logger.info(
326         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
327         req.video_path,
328         out_root,
329     )
330     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
331     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
332     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
333     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
334     overrides = []
335     sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
336     if sk is not None:
337         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
338     spec = ComputeJobSpec(
339         video_uri=req.video_path,
340         out_uri=out_root,
341         segmenter=seg_name,
342         config_overrides=tuple(overrides),
343     )

```

```

344         try:
345             result = compute.run_infer(
346                 spec
347             ) # dispatch the Cloud Run Job + bucket-poll to completion
348         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
349             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
350             return _failed(f"cloud dispatch error: {e}", True)
351         if result.returncode != 0:
352             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
353         if result.video_id and result.video_id != video_id:
354             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
355             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
356             logger.warning(
357                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
358                 "and the worker hash; ingesting under the local id.",
359                 result.video_id,
360                 video_id,
361             )
362
363         notify("ingesting (cloud_run)")
364         storage_dict = (
365             storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
366             if hasattr(storage_cfg, "model_dump")
367             else {}
368         )
369         try:
370             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
371         except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
372             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
373             return _failed(
374                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
375                 True,
376             )
377
378         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
379         segments = [
380             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
381         ]
382         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
383         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
384         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
385         provenance = {
386             "path": "cloud_run",
387             "requested": req.compute,
388             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
389             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
390             "execution": result.execution,
391             "outputs_uri": result.outputs_uri,
392         }
393         logger.info(
394             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
395             result.execution,
396             provenance["job"],
397             provenance["region"],
398             result.outputs_uri,

```

```

399         video_id,
400     )
401     return (
402         {
403             "video_id": video_id,
404             "status": "success",
405             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
406             "results": [r.model_dump() for r in val_results],
407             "play_segments": segments,
408             "compute": provenance,
409         },
410         True,
411     )
412
413
414 def _execute_processing(
415     config=None, db=None, req=None, progress=None
416 ) -> Dict[str, Any]:
417     """The full hash ? validate ? segment ? report pipeline for one video.
418
419     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
420     module-level ``global_config``/``db_instance`` (backward-compatible with existing
421     tests that pass only ``req``). Route handlers always pass explicit params; older
422     test code that calls ``_execute_processing(req)`` still works.
423
424     ``progress`` is an optional callable(stage: str) used to surface coarse job
425     progress.
426
427     Raises on unexpected internal errors ? the caller records those as a job failure
428     """
429     import backend.main as _main # call-time seam: main's globals/model stay patchable
430
431     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
432     # shift positional args ? req arrives in position 0 (config), progress in position 1
433     (db).
434
435     if config is not None and not isinstance(config, AppConfig):
436         # Old signature: _execute_processing(req, progress=None)
437         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
438             req = db
439             db = None
440         elif req is None and isinstance(config, _main.ProcessRequest):
441             req = config
442             config = None
443         # Resolve config/db from module globals when not passed explicitly
444         if config is None:
445             config = _main.global_config
446         if db is None:
447             db = _main.db_instance
448
449     notify = progress or (lambda stage: None)
450
451     notify("hashing")
452     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
453     (identity ?
454     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
455     original
456     # req.video_path (the source URI) is kept for the DB record + report; only the file-
457     reading
458     # consumers (hash / validators / segmenter) use the resolved local path.
459     local_video = ""
460     try:
461         local_video = storage.acquire_local_input(
462             req.video_path, getattr(config, "storage", None)
463         )
464     video_id = _main.compute_video_id(local_video)

```

```

458     except Exception:
459         storage.cleanup_acquired_local_input(
460             req.video_path, local_video, getattr(config, "storage", None)
461         )
462         raise
463     try:
464         existing_video = db.get_video(video_id)
465     except Exception:
466         storage.cleanup_acquired_local_input(
467             req.video_path, local_video, getattr(config, "storage", None)
468         )
469         raise
470     was_reingest = existing_video is not None
471
472     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
473     default_seg = getattr(
474         getattr(config, "indexing", None), "default_segmenter", "motion"
475     )
476     seg_name = req.segmenter or default_seg
477     segmenter_actual = None
478     segmentation_ran = False
479     val_results: List[Any] = []
480     val_context: Dict[str, Any] = {}
481     # Compute-path PROVENANCE (the validation method): which backend actually ran the
482     # "in_process" once the local segmenter is reached, set on the response as
483     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
484     # fail).
485     compute_actual: Optional[str] = None
486     response: Optional[Dict[str, Any]] = None
487     try:
488         if (
489             existing_video
490             and existing_video.get("status") == "processed"
491             and not req.force
492         ):
493             cached_segments = db.query_play_segments(video_id, {})
494             if cached_segments:
495                 logger.info(
496                     "[API] Reusing cached segmentation for video_id=%s; force=true
497                     bypasses cache.",
498                     video_id,
499                 )
500             response = {
501                 "video_id": video_id,

```

14. assistant (2026-07-05T06:57:30.415Z)

This is a key finding. The finder cited `orchestration.py:476` as being "in the CP drain" but I'm looking at `\_execute\_processing` at line 476 where `seg\_name = req.segmenter or default\_seg`. Let me verify the relationship: is `\_execute\_processing` the CP path, and where does `\_maybe\_dispatch\_remote` get `seg\_name`? Let me read the surrounding orchestration flow and find where `seg\_name` flows into the dispatch.

15. user (2026-07-05T06:57:31.914Z)

```

499         "video_id": video_id,
500         "status": "success",
501         "message": (
502             f"Reused {len(cached_segments)} cached play segments. "
503             "Pass force=true to reprocess this video."
504         ),
505         "results": existing_video.get("validation_results", []),
506         "play_segments": cached_segments,

```



```

507         "cached": True,
508         "compute": {
509             "path": "cached",
510             "requested": getattr(req, "compute", None),
511         },
512     }
513     return response
514
515     # 1. Run pluggable validators (with-context variant surfaces fprobe_meta for
the report)
516     notify("validating")
517     logger.info(f"Running validations for {req.video_path}...")
518     val_results, val_context = registry.run_all_with_context(local_video, config)
519     failures = [r for r in val_results if not r.passed]
520
521     if failures:
522         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
523         # status; it no longer destroys the video's prior good segments.
524         db.add_video(
525             video_id,
526             req.video_path,
527             "failed",
528             [r.model_dump() for r in val_results],
529         )
530         response = {
531             "video_id": video_id,
532             "status": "failed",
533             "message": _validation_failure_message(failures),
534             "results": [r.model_dump() for r in val_results],
535         }
536         return response
537
538     # 2. Run segmenter & indexer
539     logger.info("[API] Video passed checks. Segmenting and indexing...")
540     db.add_video(
541         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
542     )
543
544     segmenter_cls = segmenter_registry.get(seg_name)
545     if not segmenter_cls:
546         # Submit-time validation already 400s unknown names; this is the defensive
547         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
548         available = list(segmenter_registry.list_available().keys())
549         response = {
550             "video_id": video_id,
551             "status": "failed",
552             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
553             "results": [r.model_dump() for r in val_results],
554             "play_segments": [],
555         }
556         return response
557
558     logger.info(f"[API] Using segmenter: {seg_name}")
559     notify(f"segmenting ({seg_name})")
560     segmenter_actual = seg_name
561     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
562     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
563     play_wm, cand_wm = db.segment_watermarks(video_id)
564     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
565     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are

```

```

unchanged.
566         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
567         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
568         # they're dropped on the cloud path in this slice ? a documented follow-up.)
569         remote = _maybe_dispatch_remote(
570             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
571         )
572         if remote is not None:
573             response, segmentation_ran = (
574                 remote # `ran` = did the cloud job actually execute (report)
575             )
576             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
577         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
578         compute_actual = "in_process"
579         segmenter = segmenter_cls(db, config)
580         try:
581             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
582             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
583             with _LOCAL_COMPUTE_LANE:
584                 result = segmenter.process_video(
585                     video_id, local_video, req.player_pool, req.max_frames
586                 )
587             except Exception:
588                 # A raising segmenter must not leave half-written rows: restore the pre-run
589                 # state (same destroy-after-confirm contract as the Failure branch), then
let
590                 # the job worker record the failure.
591                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
592                 raise
593             segmentation_ran = True
594
595             if isinstance(result, Failure):
596                 logger.error(f"[API] Segmenter failed: {result.message}")
597                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
598                 response = {
599                     "video_id": video_id,
600                     "status": "failed",
601                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
602                     "results": [r.model_dump() for r in val_results],
603                     "play_segments": [],
604                 }
605                 return response
606
607             segments = result.value if hasattr(result, "value") else result
608             notify("finalizing")
609             _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
610
611             response = {
612                 "video_id": video_id,
613                 "status": "success",
614                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
.value
615                 "results": [r.model_dump() for r in val_results],
616                 "play_segments": segments,
617             }
618             return response

```



```

282         )
283         local_src = input_ref.key # resolved local path of the upload
284         staged_uri =
285         f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
286         try:
287             notify("staging upload to bucket")
288             # Thread the active storage settings so backend-specific config survives (S3
289             endpoint_url/
290             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
291             default client.
292             storage.put(
293                 local_src,
294                 staged_uri,
295                 config=(
296                     storage_cfg.model_dump() if storage_cfg is not None else None
297                 ),
298             )
299         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
300             failure, not a 500
301             logger.error(
302                 "[API] cloud-serving: failed to stage %s -> %s: %s",
303                 local_src,
304                 staged_uri,
305                 e,
306                 exc_info=True,
307             )
308             return _failed(
309                 f"cloud-serving: failed to stage the uploaded file to {input_root}:
310                 {e}",
311                 False,
312             )
313             # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
314             the L4 job
315             # DETECTs from it, and a later /api/compile re-fetches the same source (the
316             local upload is
317             # gone once this instance recycles). add_video is an UPSERT ? it only updates
318             filepath.
319             req.video_path = staged_uri
320             input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
321             db.add_video(
322                 video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
323             )
324             out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
325             if not out_root:
326                 return _failed(
327                     "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
328                     bucket) "

```

18. user (2026-07-05T06:57:39.768Z)

```

170         # < this. 0 = OFF.
171         min_players: int = Field(default=0, ge=0)
172         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
173         court_polygon: Optional[list[list[float]]] = None
174         # Net line for the person two-sided-motion split (person features only ? the shuttle
175         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
176         net_axis: Literal["x", "y"] = "y"
177         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
178
179
180     class IndexingConfig(BaseModel):
181         default_segmenter: str = "yolo_hybrid"
182         use_gpu: bool = True
183         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
184         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)

```

```

185         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
186         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
187         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
188         detector_impl: str = "auto"
189         motion_threshold: float = 0.08
190         min_segment_duration: float = 3.0
191         dead_time_merge_gap: float = 2.0
192         chunk_duration: float = 90.0
193         chunk_overlap: float = 10.0
194         detector_model: str = "fasterrcnn_resnet50_fpn"

```

19. assistant (2026-07-05T06:57:44.290Z)

Confirmed: `IndexingConfig.default_segmenter = "yolo_hybrid"` (models.py:181). And `_maybe_dispatch_remote` returns None when compute backend is None (default-OFF), meaning on the worker/non-cloud path the in-process segmenter runs. Let me now look at the top of `_maybe_dispatch_remote` to see the compute_target gate, and verify `run_infer` / worker default. Let me also check the `get_compute_backend` to confirm the CP (cloud_run target) is the one that dispatches.

20. user (2026-07-05T06:57:45.812Z)

```

200         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
201         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
202         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
203         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
204         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
205         # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
206         return (
207             {
208                 "video_id": video_id,
209                 "status": "failed",
210                 "message": msg,
211                 "results": [r.model_dump() for r in val_results],
212                 "play_segments": [],
213                 "compute": {
214                     "path": "cloud_run" if ran else "not_run",
215                     "requested": req.compute,
216                     "target": "cloud_run",
217                     "dispatched": ran,
218                 },
219             },
220             ran,
221         )
222
223         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
224         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
225         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
226         choice = (req.compute or "").strip().lower()
227         if choice and choice not in ("local", "cloud"):
228             logger.warning(
229                 "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
230                 req.compute,
231             )
232         choice = ""
233         if choice == "local":

```

```

234         return None # explicit "run on my machine" ? in-process regardless of the
server's target
235     if choice == "cloud":
236         if not _cloud_available(cfg):
237             return _failed(
238                 "cloud-serving was requested but this server isn't configured for it "
239                 "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
240                 False,
241             )
242         compute = get_compute_backend(cfg, target_override="cloud_run")
243     else:
244         compute = get_compute_backend(

```

21. user (2026-07-05T06:57:47.322Z)

```

175         if container_overrides is None
176         else tuple(container_overrides)
177     )
178
179     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
180         out_root = spec.out_uri.rstrip("/")
181         token = self._token or uuid.uuid4().hex
182         done_uri = f"{out_root}/_dispatch/{token}.done"
183         argv: List[str] = [
184             "infer",
185             "--video-uri",
186             spec.video_uri,
187             "--out-uri",
188             spec.out_uri,
189             "--done-uri",
190             done_uri,
191         ]
192         if spec.segmenter:
193             argv += ["--segmenter", spec.segmenter]
194         for kv in (*spec.config_overrides, *self._container_overrides):
195             argv += ["--set", kv]
196
197         execution = self._client.run_job(
198             self._job_name, argv, region=self._region, project=self._project
199         )
200         LOG.info(
201             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
202             self._job_name,
203             execution,
204             done_uri,

```

22. user (2026-07-05T06:57:48.027Z)

```

248         status = "failed"
249         try:
250             if failures:
251                 print(
252                     "[worker] validation FAILED: "
253                     + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
254                 )
255                 return _fail(2)
256             seg_name = args.segmenter or config.indexing.default_segmenter
257             segmenter_cls = segmenter_registry.get(seg_name)
258             if not segmenter_cls:
259                 print(
260                     f"[worker] unknown segmenter: {seg_name!r} "
261                     f"(have {list(segmenter_registry.list_available())})"
262                 )
263                 return _fail(2)

```

```
264         segmenter_actual = seg_name
265         result = segmenter_cls(db, config).process_video(
```

23. assistant (2026-07-05T06:57:51.798Z)

All the core evidence is confirmed. Let me verify two remaining points: (1) the worker's `--segmenter` arg default is None (so when CP passes it explicitly, `args.segmenter` is truthy and the worker's config default is never reached), and (2) confirm the `ProcessRequest` model has segmenter default None (so an absent field ? None, not a server-side default).

24. user (2026-07-05T06:57:53.814Z)

```
644     )
645     pi.add_argument(
646         "--out-uri",
647         required=True,
648         help="output prefix (outputs/<video_id>/? is appended)",
649     )
650     pi.add_argument(
651         "--segmenter", default=None, help="default: indexing.default_segmenter"
652     )
653     pi.add_argument(
654         "--config", default=None, help="config.json path (default: ./config.json)"
655     )
656     pi.add_argument(
657         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
658     )
659     pi.add_argument("--max-frames", type=int, default=None)
```

25. user (2026-07-05T06:57:55.400Z)

```
63:from backend.pipeline.segmenters.base import segmenter_registry
253:class ProcessRequest(BaseModel):
257:    segmenter: Optional[str] = None
266:    # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
324:    (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
332:        "default_segmenter": getattr(indexing, "default_segmenter", None),
382:        "segmenters": sorted(segmenter_registry.list_available().keys()),
383:        "default_segmenter": getattr(indexing, "default_segmenter", None),
902:    Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
953:    if req.segmenter and not segmenter_registry.get(req.segmenter):
954:        available = list(segmenter_registry.list_available().keys())
957:        detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
964:        req.segmenter,
1313:    available_segmenters = list(segmenter_registry.list_available().keys())
1315:        "--segmenter",
1316:        choices=available_segmenters,
1317:        help=f"Choose rally detection segmenter. Available: {available_segmenters}",
1520:        seg_name = args.segmenter or getattr(
1521:            getattr(global_config, "indexing", None), "default_segmenter", "motion"
1523:            segmenter_actual = None
```

[Showing results with pagination = limit: 20]

26. assistant (2026-07-05T06:58:08.849Z)

Everything is confirmed. Let me nail down the full chain:

- `ProcessRequest.segmenter: Optional[str] = None` (main.py:257) ? absent field ? None. ?
- `create\_job(..., req.segmenter, ...)` persists None (main.py:964). ?
- Worker `--segmenter default=None` (main.py in worker `\_\_main\_\_.py:651), and `seg_name = args.segmenter or config.indexing.default_segmenter` (worker:256) ? but CP always passes it

explicitly via `run\_infer` argv (cloud_run.py:192-193, always truthy since `seg\_name` on CP is never empty). ?

- The capabilities handler reads `getattr(indexing, "default\_segmenter", None)` (main.py:332, 383) ? so live capabilities reporting `yolo\_hybrid` proves the deployed config leaves the code default in place. ?

Now let me verify the one nuance the finder claimed about `default\_seg` fallback. The finder's evidence cited `orchestration.py:476` with `default\_seg = getattr(indexing, 'default\_segmenter', 'motion')`. Let me check whether there's a discrepancy ? the finder mentioned line 476 differently in the "evidence" (`getattr(indexing, 'default\_segmenter', 'motion')`) vs what I read at line 473-476. They match. The `motion` fallback only applies if `indexing` has no `default\_segmenter` attribute at all, which is not the case (the field is defined with default `yolo\_hybrid`).

Let me do one final check ? confirm the deployed config JSON snapshot doesn't pin `indexing.default\_segmenter`, and that there's no other config layer. I'll check for any env override of default_segmenter and confirm the config loading precedence.

27. user (2026-07-05T06:58:10.991Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:181:
default_segmenter: str = "yolo_hybrid"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:275:
@field_validator("default_segmenter")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\loader.py:256:# * default_segmenter "motion" ? pure-OpenCV, no torch /
GPU / weights / API key (LIGHT tier);
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\loader.py:264:
"indexing": {"default_segmenter": "motion", "use_gpu": False},
```

28. user (2026-07-05T06:58:12.377Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\config.json:26:
"default_segmenter": "gemini",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\eval_baselines\experiment_leaderboard.json:103:[Omitted long matching line]
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\eval_baselines\experiment_leaderboard.json:104:      "served_change": "config.json
indexing.default_segmenter gemini->fusion_hybrid; FusionConfig.min_crossings default 0->2",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\eval_baselines\experiment_leaderboard.json:105:      "reversible":
"indexing.fusion.min_crossings=0 (Gate-A rollback) or indexing.default_segmenter=wasb_hybrid
(full revert)",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\eval_baselines\reelcount_manifest.json:6:      "indexing.default_segmenter":
"wasb_hybrid",
```

29. assistant (2026-07-05T06:58:19.791Z)

There's an important nuance here. Let me check the `loader.py` around line 256-264 ? there's a config profile ("LIGHT tier") that sets `default\_segmenter` to "motion". The deployed config sets `profile: "cloud-serving"`. I need to check whether any profile applied at boot could set `default\_segmenter` to something other than `yolo\_hybrid`, which would change the answer. Let me examine the loader and profiles.

30. user (2026-07-05T06:58:20.731Z)

```
240      """Write a fresh default config.json.
241
242      Used by scripts/doctor.py diagnostics and the --setup CLI path.
243      Overwrites if the file exists.
244      """
245      cfg_path = Path(path) if path is not None else default_config_path()
246      cfg = AppConfig()
```



```

247         cfg_path.parent.mkdir(parents=True, exist_ok=True)
248         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
249         return cfg_path
250
251
252     # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
253     # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
skip_ai_handoff,
254     # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
255     # user gets ? a torch-free, offline, zero-setup experience:
256     # * default_segmenter "motion" ? pure-OpenCV, no torch / GPU / weights / API key
(LIGHT tier);
257     # * profile "truly-local" ? no Gemini handoff, local storage ? never surprise-
bills on
258     # ? Gemini nor returns 0 rallies on WASB-without-a-key
(08 intent);
259     # * use_gpu false ? honest for a LIGHT (no-GPU) box (motion ignores it
either way).
260     # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
261     LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
262         "sport": "badminton",
263         "deployment": {"profile": "truly-local"},
264         "indexing": {"default_segmenter": "motion", "use_gpu": False},
265     }
266
267
268     def write_light_default_config(
269         path: str | Path | None = None, *, overwrite: bool = False
270     ) -> tuple[Path, bool]:
271         """Seed the batteries-included LIGHT default config (`LIGHT_DEFAULT_CONFIG`).
272
273         Returns `(path, written)`. Unlike :func:`save_default_config` (which dumps a FULL
AppConfig
274         and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by
default ? **never
275         overwrites an existing file**, so a user's edits or the first-run wizard's choices
are preserved
276         (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
dirs.
277         """
278         cfg_path = Path(path) if path is not None else default_config_path()
279         if cfg_path.exists() and not overwrite:
280             return cfg_path, False
281         cfg_path.parent.mkdir(parents=True, exist_ok=True)
282         cfg_path.write_text(
283             json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n", encoding="utf-8"
284         )
285         return cfg_path, True
286
287
288     # Backwards-compatible alias used by some older call sites during transition.
289     def get_default_config() -> dict[str, Any]:
290         """Return defaults as a plain dict (for transitional use only)."""
291         return _get_builtin_defaults()
292

```

31. assistant (2026-07-05T06:58:27.152Z)

This `LIGHT\_DEFAULT\_CONFIG` is only used by `write\_light\_default\_config` to seed a fresh file on first-run for the local appliance ? it is NOT applied to the deployed cloud-serving CP (whose config.json is base64-injected at boot and sets `profile: "cloud-serving"`, not `truly-local`). So it doesn't override the deployed default. Now let me check the profile presets to make sure `cloud-serving` profile doesn't pin `default\_segmenter`.

32. user (2026-07-05T06:58:29.203Z)

```
4:config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
34:     "cloud-serving": "cloud_run",
45:     "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
51:     "cloud-serving": {
52:         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
65:         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
123:     - ``K_SERVICE`` set (Cloud Run)      ? ``"cloud-serving"``
129:     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
134:         return "cloud-serving"
```

33. assistant (2026-07-05T06:58:34.129Z)

Let me see the full `cloud-serving` preset block to confirm it does NOT set
`indexing.default\_segmenter`.

34. user (2026-07-05T06:58:34.878Z)

```
40     # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41     # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54             # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55             # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56             # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57             "indexing": {
58                 "detector_impl": "native",
59                 "skip_ai_handoff": False,
60                 "wasb": {"batch_size": 64, "prefetch_batches": 4},
61             },
62             "storage": {"backend": "gcs"},
63         },
64         "cpu-mock": {
65             "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
66             # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
67             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
68             "storage": {"backend": "local"},
69         },
70     }
71
72
73     def is_known_profile(profile: str) -> bool:
74         """True for a non-empty, recognised profile name."""
75         return bool(profile) and profile in PROFILE_PRESETS
76
77
```

```

78     def preset_for(profile: str) -> dict[str, Any]:
79         """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
80         module-level table), or ``{}`` for ``""`` / an unknown profile."""
81         return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
82
83
84     def _truthy(val: Optional[str]) -> bool:
85         """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
86         if val is None:
87             return False
88         return val.strip().lower() in {"1", "true", "yes", "on"}
89
90
91     def _cuda_available() -> bool:
92         """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
93         ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
94         try:
95             # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
96             # and optional). Only reached when a caller opts into probe_cuda.
97             import torch
98
99             return bool(torch.cuda.is_available())
100         except Exception:
101             return False
102
103
104     def probe_cuda_available() -> bool:
105         """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
106
107         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
108         that *wants* the GPU heads-up can pass the result in without the startup module
importing
109         torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
110         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
111         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
112         error that could false-fail."""
113         return _cuda_available()
114

```

35. assistant (2026-07-05T06:58:48.530Z)

This is the decisive confirmation. The `cloud-serving` profile preset (presets.py:51-63) sets `indexing.detector\_impl`, `skip\_ai\_handoff`, and `wasb.batch\_size/prefetch` ? but it does ****NOT**** set `indexing.default\_segmenter`. So even with the profile preset applied, `default\_segmenter` falls back to the code default `yolo\_hybrid` (models.py:181), exactly matching the live capabilities value.

This actually strengthens the finder's conclusion beyond the "config.json doesn't pin it" argument: even the profile machinery for cloud-serving does not pin the segmenter. And comment line 53 explicitly says "Linux L4 (cuda); Gemini handoff ON" ? the cloud-serving profile was designed around the WASB/detector path but never pins the segmenter name itself, so `yolo\_hybrid` (the code default) is what resolves.

I have now independently verified every load-bearing link in the chain. Let me confirm there's no env-var override of default_segmenter in the loader precedence.

36. user (2026-07-05T06:58:50.725Z)

```
33:# ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-  
relative) when  
36:# from a later RALLY_APP_HOME.)  
62:     env_profile = os.environ.get("DEPLOYMENT_PROFILE")
```

37. assistant (2026-07-05T06:59:08.480Z)

No env-var override of `default\_segmenter` exists (only `DEPLOYMENT\_PROFILE` for profile selection, which as shown doesn't pin the segmenter). The chain is fully verified.

Let me summarize my independent verification and try to refute the finder's net verdict ? I could find no refutation. Every link holds:

1. **SPA omits segmenter** (client.ts:98-102 body has only `video\_path, locale, compute`; sole caller useIngestJob.ts:259 passes 4 args, no segmenter; grep proves `segmenter` never appears in the ingest/process path, only in capabilities-type/display). ? CONFIRMED
2. **`req.segmenter` arrives None** (ProcessRequest.segmenter Optional[str]=None, main.py:257). ?
3. **CP resolves `seg\_name = req.segmenter or default\_seg`** where `default\_seg = getattr(indexing, 'default\_segmenter', 'motion')` (orchestration.py:473-476). ?
4. **Deployed default is `yolo\_hybrid`**: code default (models.py:181) is `yolo\_hybrid`; neither deployed config.json NOR the `cloud-serving` profile preset (presets.py:51-63) pins `indexing.default\_segmenter`; no env override exists; live capabilities reporting `yolo\_hybrid` (via getattr at main.py:332/383) proves this empirically. ?
5. **CP forwards `seg\_name` explicitly** to worker as `--segmenter yolo\_hybrid` (orchestration.py:341 ? spec.segmenter; cloud_run.py:192-193 always-truthy). ?
6. **Worker default never reached**: `--segmenter default=None` (worker:651), `seg\_name = args.segmenter or config.indexing.default\_segmenter` (worker:256), but CP always passes it explicit so worker's own default fallback is dead. ?

One minor overstatement in the finder to note: the finder's `deployed\_vs\_master` says it "read the deployed config.json only from the shared session context (not fetched fresh)" ? that caveat is honest and correct; I likewise could not fetch the live running config in this trace, but the code+capabilities evidence makes the conclusion airtight regardless. The finder's fix options and open questions are sound (particularly the open question about whether yolo_hybrid even produces valid results on the worker ? that's a real, unverified downstream risk, not part of the segmenter-selection claim itself).

38. user (2026-07-05T06:59:34.108Z)

Structured output provided successfully